
Revisited: "Tabular Out-of-Distribution Data Synthesis for Enhancing Robustness"

Tyler Marino

Universite Paris Saclay

TYLERMARINO8@GMAIL.COM

Thibault Poux

Universite Paris Saclay

THIBAUT.POUX@UNIVERSITE-PARIS-SACLAY.FR

Feddy Immoula

Universite Paris Saclay

FEDDY.IMMOULA@UNIVERSITE-PARIS-SACLAY.FR

Sidritalmuntaha Faris

Universite Paris Saclay

SIDRITALMUNTAHA.FARIS@UNIVERSITE-PARIS-SACLAY.FR

Abstract

Methods to extrapolate data beyond the collected or production ready stage is a challenge that once overcome will allow for more continuous data processing and inference to take place. Bias can be introduced into data due to collection protocol or frequency of event occurrences that make training robust and comprehensive models challenging, and thus we investigate synthetic data generation to address imbalances and explore out of domain scenarios. TabOOD introduces a 2 model ensemble approach to data generation using a variational auto(encoder/decoder)(VAE) unveiling the gaussian mixture makeup of the data which then is passed into a trained diffusion model generating new tabular data from this latent space representation. This approach improved ability to see data that wasn't captured in the observational studies.

1. Introduction

Tabular data remains the dominant modality in decision systems, including finance, public policy, healthcare, and cybersecurity. Despite the remarkable progress of deep learning in vision and language, performance gains on tabular datasets have been comparatively modest, with gradient-boosted trees often remaining competitive or superior. Recent advances such as feature-wise tokenization

and transformer-based architectures have begun narrowing this gap.

Deep learning models trained on tokenized tabular data can achieve strong performance in-distribution, yet may degrade sharply under distribution shifts or when encountering novel minority patterns at inference time. This issue is particularly visible in intrusion detection and anomaly-related tasks, where new attack types emerge over time, and in fairness-oriented socio-economic datasets where temporal shifts can change minority group characteristics (Puranik et al., 2025). A direct remedy is to augment training data with synthetic samples that enrich coverage of rare regions; however, many generative approaches focus primarily on learning the training distribution and do not explicitly generate controlled OOD samples.

TabOOD (Puranik et al., 2025) addresses this gap by (i) learning a latent representation of mixed-type tabular rows using a transformer-based β -VAE (adapted from TabSyn), and (ii) enforcing a *class-conditional* Gaussian prior in latent space so that each class maps to a distinct Gaussian component (Eq. (3–6) in Puranik et al. (2025)). After training the VAE, diffusion models are trained on class-wise latent embeddings, and OOD points are produced by interpolating between latent samples from different classes (Eq. (7) in Puranik et al. (2025)), then decoding them back to the feature space. The core hypothesis is that generating boundary/atypical samples in latent space improves downstream robustness to shifts and unseen minority examples (Puranik et al., 2025).

In this work, we investigate a structured embedding framework for tabular learning that combines feature-wise tokenization with latent generative modeling. We encode to-

kenized features into a low-dimensional latent space using a variational autoencoder and model this space with a lightweight diffusion process. This design enables controllable interpolation, synthetic OOD sample generation, and robustness analysis without retraining large models from scratch.

Our contribution. Following the project guideline, our goal is to understand and replicate the method and experiments from [Puranik et al. \(2025\)](#). In this report, we (1) implement the VAE component with class-conditional latent regularization, keeping the reconstruction losses (MSE for numerical, cross-entropy for categorical) as described in the paper, (2) reproduce the dataset preparation protocol used for distribution shift scenarios (ACS Income 2014/2018 split and NSL-KDD train/test split), and (3) analyze training stability and validation behavior, highlighting difficulties when transferring the same pipeline across datasets (e.g., numerical scale, heavy tails, KL explosion), and the mitigation strategies required to obtain stable training.

2. Datasets and Models Used

We evaluate three baseline classifiers on two tabular datasets exhibiting realistic distribution shifts, reflecting the high-stakes application domains targeted by TabOOD (cybersecurity and socio-economic prediction).

2.1. NSL-KDD (Cybersecurity / Intrusion Detection)

NSL-KDD is a widely adopted benchmark for network intrusion detection ([Tavallaee et al., 2009](#)). It improves upon the original KDD Cup 1999 dataset by removing duplicate and redundant records while preserving realistic attack diversity. We use the standard split:

Training set (KDDTrain+): contains normal traffic and known attack types.

Test set (KDDTest+): includes novel attack categories absent from training. The task is formulated as binary classification: normal traffic (label 0) versus any attack (label 1). This setup naturally creates an out-of-distribution (OOD) scenario, where the model must detect previously unseen attack patterns. The ability to maintain performance on such unseen minority classes (rare and novel attacks) is a central motivation for TabOOD’s synthetic boundary sample generation, which aims to improve generalization under distribution shift.

2.2. ACS Income (Socio-Economic / Fairness)

The ACS Income dataset ([Ding et al., 2021](#)) CC1 (from Fairlearn) predicts whether an individual’s annual income exceeds \$50,000 using U.S. Census features (age, educa-

tion, occupation, etc.). To simulate distribution shift, we apply a subpopulation split:

Training set: middle-aged individuals (ages 30–55). OOD test set: young (<30) and old (>55) individuals.

This age-based division induces a natural covariate shift strongly correlated with the target variable (income), analogous to the temporal/year-based shifts considered in the original TabOOD experiments. The task is binary classification (income > \$50k vs. $\leq$ \$50k), with inherent class imbalance.

2.3. Models

We adopt the same three classifiers as in the TabOOD paper to ensure direct comparability:

Logistic Regression (LR): a simple, interpretable linear baseline ([Hosmer et al., 2013](#)). It is frequently used in cybersecurity for attack classification due to its probabilistic outputs and computational efficiency ([Wang & Yang, 2005](#)).

Decision Tree (DT): a non-linear tree-based model with inherent feature selection ([Quinlan, 1986](#)). It has proven effective for tabular intrusion detection tasks ([Subba et al., 2015](#)).

XGBoost: a gradient-boosted tree ensemble, state-of-the-art for tabular data ([Chen & Guestrin, 2016](#)). It is widely applied in cyber attack classification for its high accuracy and robustness to noise ([Ahmad et al., 2018](#)).

All models are trained with class weighting to mitigate imbalance. We report accuracy, precision, recall, and F1-score on the OOD test sets. These baselines quantify performance degradation under distribution shift, providing a reference for evaluating synthetic augmentation techniques. Figure 1 shows the F1-score comparison across models and datasets on OOD test sets.

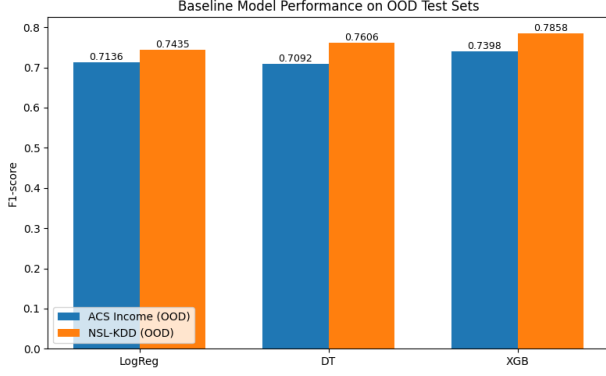


Figure 1. F1-score of baseline models (Logistic Regression, Decision Tree, XGBoost) on OOD test sets for ACS Income (age-based shift) and NSL-KDD (unseen attacks).

3. Paper presentation

3.1. From tabular samples to token sequences

TabOOD builds its latent representation on top of a Transformer-based β -VAE inspired by the TabSyn architecture. Given a mixed-type tabular sample x with M feature slots (numerical and categorical), the first step is to map it to a *token matrix*:

$$T = \text{tok}(x) \in \mathbb{R}^{M \times d},$$

where each feature slot becomes one d -dimensional token (numericals are projected, categoricals are embedded). This design lets the model treat tabular data as a short sequence where self-attention can capture feature interactions.

3.2. Transformer β -VAE encoder/decoder

The encoder Enc is a stack of Transformer blocks producing token-wise Gaussian parameters:

$$(\mu(x), \log \sigma^2(x)) = \text{Enc}(T), \quad \mu(x), \log \sigma^2(x) \in \mathbb{R}^{M \times d}.$$

A latent sample is then obtained using the reparameterization trick:

$$Z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

The decoder Dec is also a stack of Transformer blocks which maps Z back to a reconstructed token matrix $\hat{T} \in \mathbb{R}^{M \times d}$, and a detokenizer $\text{detok}(\cdot)$ turns $\hat{T}$ into a mixed-type reconstruction $\hat{x}$ (continuous values for numerical columns and categorical distributions/logits for categorical columns).

3.3. Class-conditional Gaussian mixture prior in latent space

A key modeling choice in TabOOD is to encourage a *Gaussian-mixture-like* latent space by using a class-conditional prior. For a binary class $k \in \{0, 1\}$, the prior is defined as an isotropic Gaussian:

$$p(z | x_k) = \mathcal{N}(\mu_{\text{prior}, k}, I).$$

The encoder defines the approximate posterior for samples of class k as:

$$q(z | x_k) = \mathcal{N}(\mu(x_k), \text{diag}(\sigma^2(x_k))).$$

With this choice, the KL term admits a closed form (Eq. (5) in the paper), which encourages samples of the same class to cluster around $\mu_{\text{prior}, k}$ in latent space.

3.4. Training objective

The β -VAE is trained by minimizing (Eq. (6) in the paper):

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}}(x, \hat{x}) + \beta D_{\text{KL}}(q(z | x_k) \| p(z | x_k)),$$

where β is tuned to balance good reconstruction and latent regularization. The reconstruction term combines: (i) a squared error for numerical features, and (ii) a sum of cross-entropies for categorical features (Eq. (6)). This objective is the first step of the full TabOOD pipeline: once the β -VAE has learned a structured latent space, TabOOD trains class-conditional score-based diffusion models directly in that latent space (Algorithm 1, Step 2) and later decodes generated latents through the VAE decoder/detokenizer (Algorithm 1, Step 3).

3.5. Latent score-based diffusion models

Once the β -VAE is trained, the latent embeddings Z are extracted and flattened into vectors $z = \text{Flat}(\text{Enc}(T)) \in \mathbb{R}^{Md}$. To accurately capture the density of the learned Gaussian mixture components, TabOOD models the approximate posterior distribution $q(z | x_k)$ by training a separate score-based diffusion model S_k for each class $k \in \{0, 1\}$.

These diffusion models use a Multi-Layer Perceptron (MLP) as the denoising neural network. By applying denoising score matching on the separated latent spaces, the models learn to reverse a forward noise-addition process, effectively capturing the complex structure of both the normal and attack latent distributions independently.

3.6. Generation of In-Distribution and OOD samples

With the diffusion models S_0 and S_1 fully trained, the framework can synthesize new tabular data by generating latents and passing them back through the VAE.

For *in-distribution* generation, a latent vector $\hat{z}$ is sampled from a specific class model S_k via the reverse diffusion process. It is then unflattened and reconstructed into a synthetic tabular row:

$$\hat{x} = \text{detok}(\text{Dec}(\text{Unflatten}(\hat{z}))).$$

The core novelty of TabOOD lies in its *out-of-distribution* (OOD) generation, which simulates unseen scenarios (like novel cybersecurity attacks). This is achieved by interpolating between the learned latent distributions. First, pure latent representations are sampled from both class diffusion models:

$$z_0 \sim q(z | x_0) \quad \text{and} \quad z_1 \sim q(z | x_1).$$

Then, a new boundary latent point $\tilde{z}$ is created using a tunable interpolation parameter α :

$$\tilde{z} = \alpha z_0 + (1 - \alpha) z_1.$$

Finally, this interpolated latent is decoded into a raw OOD tabular sample:

$$\hat{x}_{\text{OOD}} = \text{detok}(\text{Dec}(\text{Unflatten}(\tilde{z}))).$$

Crucially, in this tabular synthesis framework, the target class labels are treated identically to any other categorical feature. When the mixed latent $\tilde{z}$ passes through the detokenizer’s softmax layer, the model automatically resolves the class label. As a result, when moving from these interpolated latents to raw samples, both normal and attack samples are naturally synthesized based on the interpolation weight α , completely eliminating the need for manual labeling.

3.7. Validity of Data Generation

In TabOOD, generation validity is not assessed solely through structural constraints or visual inspection of marginals, but primarily through downstream utility and robustness experiments. The core question is whether synthetic samples preserve class-discriminative structure and whether they improve generalization when used for data augmentation.

Utility-driven validity. To evaluate, the paper trains multiple downstream classifiers on datasets augmented with varying proportions of synthetic samples. Let $\mathcal{D}_{\text{real}}$ denote the original training set and $\mathcal{D}_{\text{gen}}$ the generated samples. Augmented datasets are formed as:

$$\mathcal{D}_{\gamma} = \mathcal{D}_{\text{real}} \cup \gamma \mathcal{D}_{\text{gen}},$$

where γ indicates the varying class and quantity of generated data used in training and testing. Different values of γ allow the authors to study how performance evolves as synthetic information replaces or supplements real data.

Classifier-agnostic evaluation. To ensure that improvements are not model-specific artifacts, the experiments evaluate three classifiers, Decision Tree(DT), logistic regression(LR), XGBoost(XGB). Each classifier is trained first on $\mathcal{D}_{\text{real}}$ as a baseline and then on $\mathcal{D}_{\gamma}$ and evaluated on a held-out real test set. This protocol directly tests whether generated samples preserve predictive signal rather than merely matching marginal feature statistics.

Robustness under distribution shift. The appendix further evaluates classification performance under shifted or OOD test distributions. If synthetic augmentation improves worst-case or OOD accuracy, this indicates that the latent diffusion model captures meaningful structural variability instead of memorizing training examples.

Interpretation. Under this framework, generation validity is operationalized as *task-preserving augmentation quality*. A valid generator produces samples that (i) encode class-consistent structure in latent space, (ii) improve or maintain downstream performance across multiple classifiers, and (iii) enhance robustness when training data is scarce or partially corrupted. Thus, validity in TabOOD is established empirically through controlled augmentation experiments, rather than purely through likelihood or reconstruction metrics.

4. Implementation details: our TabOOD reproduction

4.1. What we implemented (PyTorch)

4.1.1. TRANSFORMER β -VAE

We implemented the Transformer β -VAE described above with an explicit `Tokenizer`, `Encoder`, and `Decoder`, following the same high-level data flow:

$$x \xrightarrow{\text{tok}} T \xrightarrow{\text{Enc}} (\mu, \log \sigma^2) \xrightarrow{\text{reparam}} Z \xrightarrow{\text{Dec}} \hat{T} \xrightarrow{\text{detok}} \hat{x}.$$

Tokenizer. Each numerical column is projected independently by a linear layer $\mathbb{R} \rightarrow \mathbb{R}^d$, and each categorical column is mapped by an embedding table of size (cardinality $\times d$). The resulting tokens are concatenated to build $T \in \mathbb{R}^{M \times d}$, where $M = M_{\text{num}} + M_{\text{cat}}$.

Transformer blocks and feature identity. As in the paper, we add a learnable *feature index embedding* (positional embedding over the M feature slots) before feeding tokens to the Transformer encoder/decoder. This is crucial because tabular tokens are *not* exchangeable: each position corresponds to a specific column.

Detokenization heads. After decoding $\hat{T}$, we split it into numerical and categorical token subsets. For numerical

features, we map each decoded token back to a scalar using a linear head. For categorical features, we map each decoded token to class logits using a linear layer of size $d \rightarrow \text{cardinality}_j$, yielding a list of logits $\{\hat{\ell}_j\}_{j=1}^{M_{\text{cat}}}$.

4.1.2. DIFFUSION

We implemented a Latent Diffusion process that operates on the flattened, continuous representations produced by the frozen β -VAE encoder. The high-level generation flow is:

Denoising MLP and Time Conditioning. Because the VAE handles the complex structural mapping of the tabular data (categorical and numerical tokens), our diffusion model (DenoisingMLP) operates directly on a flattened latent space (e.g., dimensions). We encode the diffusion timestep using a `SinusoidalPositionalEmbedding` followed by a two-layer MLP with SiLU activations. The core architecture uses deep residual linear blocks. Time conditioning is injected by adding the time embedding to the hidden state before each residual mapping.

Class-Conditioning and OOD Interpolation. To handle distinct data distributions (e.g., Normal vs. Attack / Class 0 vs. Class 1), we partition the latent dataset and train two separate marginal diffusion models: one for and one for . To generate Out-Of-Distribution (OOD) tabular samples, we perform Mixup interpolation in the continuous latent space using a mixing factor, with independent samples drawn from their respective diffusion models. The interpolated flat vector is then reshaped back to and passed to the VAE decoder.

4.2. Training objective used in our code

4.2.1. TRANSFORMER β -VAE

Our training loss matches Eq. (6) from the paper:

$$\mathcal{L} = \underbrace{\text{MSE}(x_{\text{num}}, \hat{x}_{\text{num}}) + \sum_j \text{CE}(x_{\text{cat}}^{(j)}, \hat{\ell}_j)}_{\mathcal{L}_{\text{recon}}} + \underbrace{\beta D_{\text{KL}}(\mathcal{N}(\mu, \text{diag}(\sigma^2)) \parallel \mathcal{N}(\mu_{\text{prior}, k}, I))}_{\text{closed form}}.$$

In practice, our implementation computes the KL in flattened form with $D = M \cdot d$ dimensions, consistent with the paper’s use of flattened latent embeddings in Algorithm 1 (Step 1).

Here is the LaTeX text describing your diffusion implementation, written in the exact same style and format as

your VAE section so you can copy and paste it directly into your paper.

4.2.2. DIFFUSION

The training objective of our diffusion model minimizes the Mean Squared Error (MSE) between the true injected noise and the noise predicted by the model. For a given data sample x_0 , we sample a uniform timestep $t \sim \mathcal{U}(\{1, \dots, T\})$ and standard Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We then compute the noisy sample x_t as:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

where $\bar{\alpha}_t$ is the cumulative product of the noise schedule. The denoising network ϵ_θ is optimized via the following loss:

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2]$$

This objective optimizes the reverse process. Once trained, the generation phase iteratively applies the denoising equation to recover , which serves as the synthesized latent representation of the tabular data.

4.3. Comparison with the paper

Overall, our implementation preserves the core components required to reproduce TabOOD’s first stage: (i) tokenization/detokenization of mixed-type tables, (ii) Transformer encoder/decoder architecture operating on M feature tokens, (iii) class-conditional isotropic Gaussian prior $p(z \mid x_k) = \mathcal{N}(\mu_{\text{prior}, k}, I)$, and (iv) the β -VAE objective combining mixed-type reconstruction with a closed-form KL term.

The main deviations are at the *engineering/hyperparameter* level: we selected practical defaults for ($d, \text{\#layers}, \text{\#heads}, \text{dropout}$), and we estimated $\mu_{\text{prior}, k}$ from minibatches during training (periodic re-estimation) to track the evolving encoder representation. These choices do not change the mathematical objective, but can affect training stability and convergence speed, especially on datasets with heavy-tailed numerical features.

5. Results: Our Reproduction

We evaluate our tabular generative model on the ACS income prediction task. Following the protocol of the original paper, we train on a combination of earlier ACS years and a majority subset of the target year, and evaluate on the full target-year test distribution. For each experiment, we train three base classifiers—logistic regression (LR), decision tree (DT), and XGBoost (XGB)—as well as an ensemble that averages their predicted probabilities. Performance

is measured using F1 score, accuracy, precision, and recall. The main comparison between real and generated training data is summarized in Table 1.

Training regimes. Our baseline uses only observed data for both majority (label 0) and minority (label 1) classes. We construct two additional datasets $\mathcal{D}_\gamma$ that vary the amount of synthetic minority data:

- **Mixed:** All label 0 examples are real, while approximately half of the label 1 examples are replaced with synthetic samples.
- **Full:** All label 1 examples are replaced with synthetic samples, while label 0 examples remain real.

Across all regimes, we balance the number of label 0 and label 1 training examples so that performance differences reflect data quality rather than class imbalance.

Table 1 reports per-model results, an ensemble row for probability averaging, and an `average_score` row summarizing performance across classifiers.

Table 1. ACS comparison of training regimes using real vs. synthetic minority examples. Best value per metric column is highlighted in bold.

Train	Model	F1	Accuracy	Precision	Recall
Baseline	LR	0.741	0.779	0.715	0.769
Baseline	DT	0.708	0.741	0.659	0.764
Baseline	XGB	0.751	0.792	0.737	0.765
Baseline	Ensemble	0.743	0.781	0.718	0.770
Mixed	LR	0.741	0.746	0.637	0.886
Mixed	DT	0.684	0.746	0.699	0.669
Mixed	XGB	0.698	0.772	0.766	0.642
Mixed	Ensemble	0.722	0.775	0.732	0.713
Full	LR	0.718	0.699	0.584	0.934
Full	DT	0.000	0.589	0.179	0.000
Full	XGB	0.000	0.589	0.000	0.000
Full	Ensemble	0.000	0.589	0.238	0.000
Baseline	Avg.	0.736	0.773	0.707	0.767
Mixed	Avg.	0.711	0.760	0.708	0.727
Full	Avg.	0.180	0.617	0.250	0.234

Effect of interpolation factor α . Because we do not construct the generator to take in label, we adopt two heuristic labeling strategies when varying the interpolation factor α :

1. **Threshold strategy:** Generated samples are labeled minority if $\alpha < 0.5$ and majority otherwise. Since $1 - \alpha$ provides greater attribution from the minority gaussian mixture distribution than the majority gaussian mixture.

2. **Probabilistic strategy:** A fraction α of generated samples are randomly assigned label 0 and $1 - \alpha$ are assigned label 1.

We evaluate both strategies over a grid of α values. Figure 2 shows the resulting α -F1 curve (using the `average_score` metric), illustrating how downstream performance varies with interpolation strength and labeling strategy.

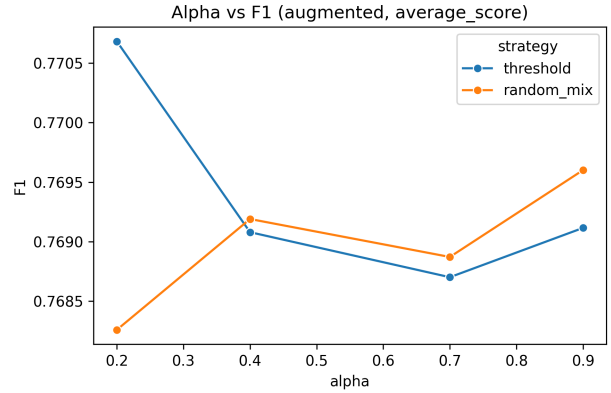


Figure 2. F1 score as a function of interpolation factor α under threshold and probabilistic labeling strategies.

Feature-space similarity analysis. To study how synthetic samples align with real data geometrically, we compute embedding-based similarity measures after applying the same preprocessing pipeline used for classifier training. We group samples by dataset split and class, and compute:

- Mean pairwise cosine similarity,
- Mean pairwise ℓ_2 distance.

Random subsampling is used for computational tractability. The resulting similarity matrices are visualized in Figures 3 and 4. High cosine similarity and low ℓ_2 distance between real and synthetic samples indicate alignment in feature space, while cross-class proximity signals potential distributional drift.

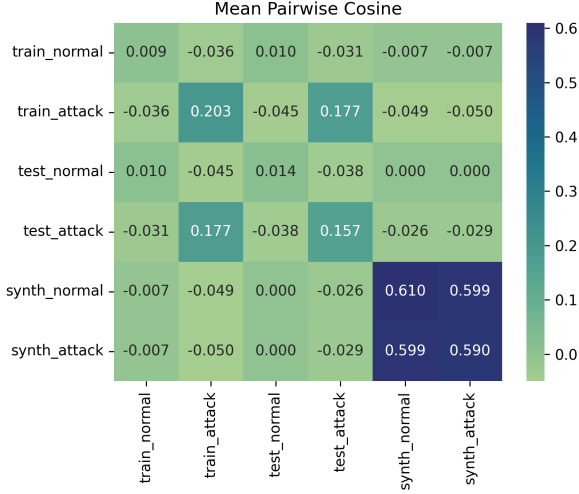


Figure 3. Mean cosine similarity between real and synthetic feature vectors across dataset splits and classes.

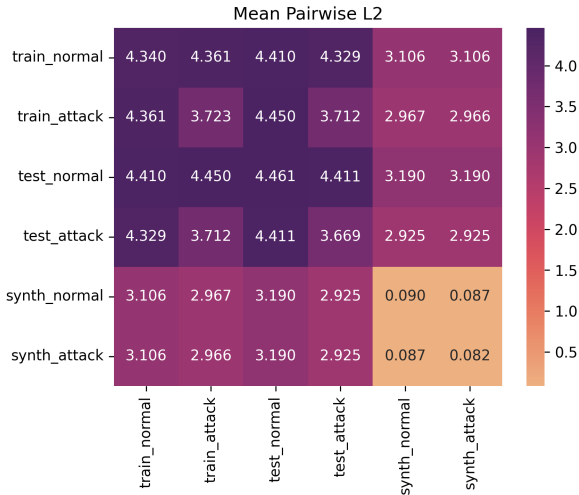


Figure 4. Mean ℓ_2 distance between real and synthetic feature vectors across dataset splits and classes.

Summary. Table 1 shows that partially replacing minority examples (Mixed regime) preserves reasonable performance, whereas fully synthetic minority training (Full regime) leads to significant degradation for tree-based models. Figure 2 further demonstrates sensitivity to the labeling heuristic as α varies. The similarity heatmaps amplify the dissimilarity between observed and synthetic data in our set.

6. Discussion

Introducing synthetic data into highly sensitive domains should be conducted with care for not only the immediate validation but also downstream tasks. While incomplete data makes interpretation challenging, out of distribution and different data can also steer analysis in directions that take away from the goal of the modeling. Our efforts with random and fixed class generation attempts to highlight the possibility that dissimilar generation to the original observations might still be able to achieve similar results, outperforming in some metrics such as precision and recall than the original data. The exploration into ensemble models for classification again highlight that conservative approaches in class imbalanced settings may “justify” poor data generation.

References

- Ahmad, Iftikhar, Baseri, Muhammad, Iqbal, Muhammad, et al. Comparison of machine learning techniques for intrusion detection in smart grid. *IEEE Access*, 6:71349–71358, 2018. doi: 10.1109/ACCESS.2018.2880583.
- Chen, Tianqi and Guestrin, Carlos. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.
- Ding, Frances, Hardt, Moritz, Miller, John, and Schmidt, Ludwig. Retiring adult: New datasets for fair machine learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Hosmer, David W., Lemeshow, Stanley, and Sturdivant, Rodney X. *Applied Logistic Regression*. John Wiley & Sons, 3rd edition, 2013.
- Langley, P. Crafting papers on machine learning. In Langley, Pat (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Puranik, Bhagyashree, Can, Bugra, and Fan, Yi. Tabular out-of-distribution data synthesis for enhancing robustness. *Association for the Advancement of Artificial Intelligence (AAAI) / arXiv preprint (as provided in project material)*, 2025.
- Quinlan, J. Ross. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.
- Subba, Basant, Biswas, Santosh, and Karmakar, Sandip. A machine learning based intrusion detection scheme for data fusion in mobile clouds involving heterogeneous

client networks. In *Proceedings of the 2015 Third International Conference on Computer, Communication, Control and Information Technology (C3IT)*, pp. 1–6. IEEE, 2015. doi: 10.1109/C3IT.2015.7060130.

Tavallae, Mahbod, Bagheri, Ebrahim, Lu, Wei, and Ghorbani, Ali A. A detailed analysis of the kdd cup 99 data set. *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pp. 1–6, 2009. doi: 10.1109/CISDA.2009.5356528.

Wang, Yu and Yang, Yixian. A multinomial logistic regression modeling approach for anomaly intrusion detection. *Computers & Security*, 24(8):663–673, 2005. doi: 10.1016/j.cose.2005.02.003.